GPU Infrastructure Lab

Overview:

This lab introduces you to GPU infrastructure through hands-on exploration. You will connect to a real GPU environment, examine its hardware properties, and run experiments that demonstrate why GPUs are essential for AI workloads. By the end of the lab, you will have directly observed the difference in computational speed between a CPU and a GPU and loaded a small neural network model onto GPU memory.

The lab runs in a Jupyter Notebook environment.

Language: Python3 with PyTorch

Platform: Google Colab (Welcome To Colab - Colab)

Step 0: Access the GPU environment

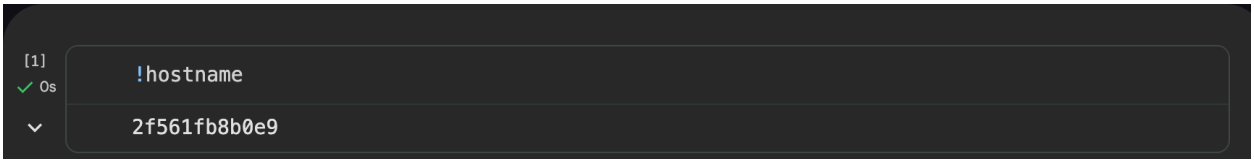
- Create a notebook in Google Colab.
- Click on the Runtime menu at the top of the page.
- Select Change runtime type from the dropdown list.
- Under the Hardware accelerator dropdown, select GPU (the free tier typically provides an Nvidia T4 GPU).
- Click Save. The notebook will automatically refresh and connect to a GPU instance.

Record your output after each of the following steps (screenshot or paste output)

Step 1: Identify your computer

The host name command returns the name of the machine your notebook is running on. This is especially useful in a shared GPU cluster where many different machines may be available.

```
!hostname
```

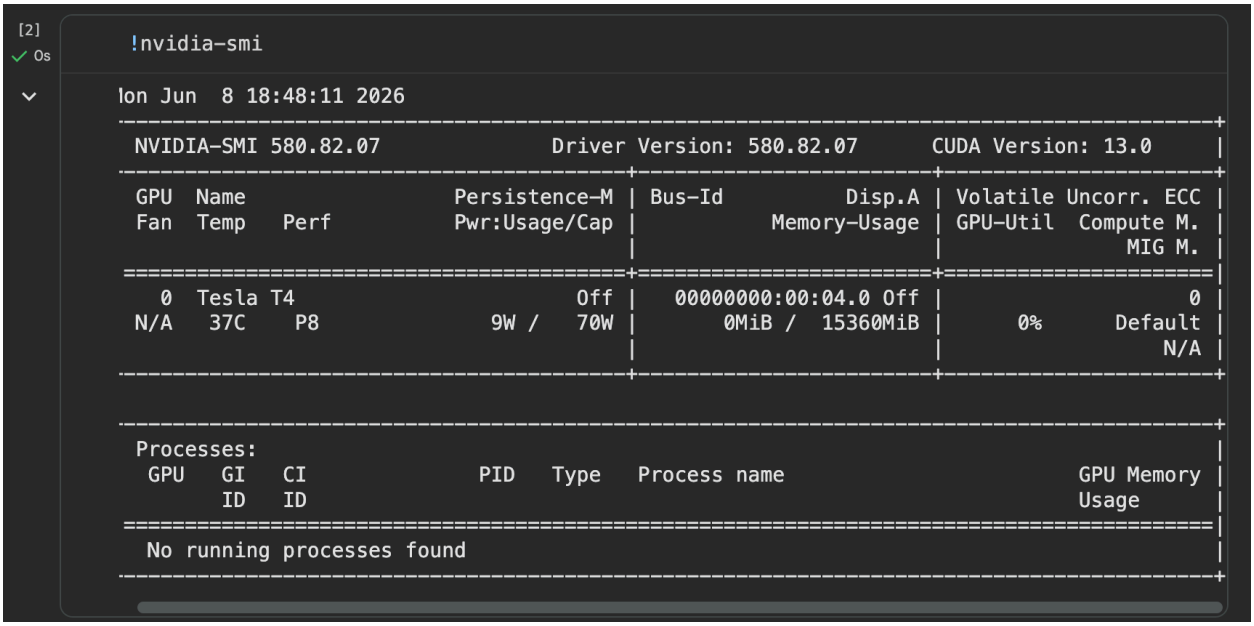


Expected output: *On Google Colab you will see a randomly generated machine name. On a JupyterHub cluster you will see the name of the specific GPU node assigned to your session.*

Step 2: Inspect the GPU(s)

nvidia-smi stands for NVIDIA System Management Interface. It is a command-line tool that queries the GPU driver and returns detailed information about every GPU available on the machine. This is the standard tool used by engineers and researchers to check GPU status in production environments.

```
!nvidia-smi
```



Expected output: *GPU-Util will be 0% at this stage because we have not run any workloads yet. You will see this number increase in later steps when we run matrix operations*

Output field	What it means
`GPU Name`	The model of GPU — e.g. Tesla T4, A100, or H100
`Driver Version`	The version of the NVIDIA driver installed on the machine
`CUDA Version`	The highest CUDA version supported by this driver
`Memory-Usage`	How much GPU memory (VRAM) is currently in use vs total available
`GPU-Util`	Percentage of GPU compute currently being used (0% means idle)
`Temp`	Current GPU temperature in Celsius

Step 3: Verify PyTorch and CUDA availability

PyTorch is the most widely used deep learning framework for AI research and production. Before running any AI workloads, let's confirm that PyTorch is installed and that it can detect and communicate with the GPU through CUDA (NVIDIA's parallel computing platform)

```
import torch

print(torch.__version__)

print(torch.cuda.is_available())
```

Expected output: *If torch.cuda.is_available () it returns true, and you will see the version. If not, it returns False, PyTorch cannot see the GPU. This usually means the runtime is CPU-only. Check your runtime settings before continuing.*

Step 4: Query GPU hardware properties

Now that we have confirmed the GPU is available, we can query its hardware specifications directly from Python using PyTorch's CUDA API. These properties determine what size models the GPU can run and how fast it can process AI workloads.

```
if torch.cuda.is_available():
    device = torch.cuda.get_device_properties(0)
    print("GPU Count:", torch.cuda.device_count())
    print("GPU Name:", device.name)
    print("Total Memory (GB):", round(device.total_memory / (1024**3), 2))
    print("Multiprocessors:", device.multi_processor_count)
```

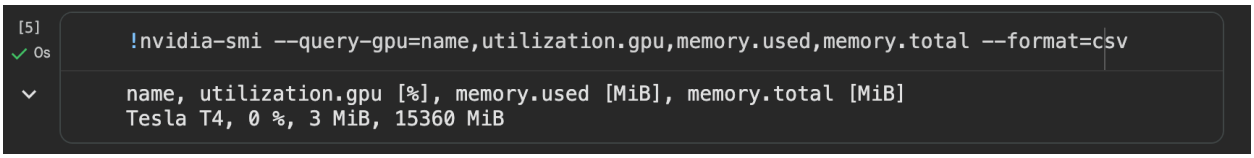
Expected output:

Output field	What it means
`GPU Count`	Number of GPUs available in this session
`GPU Name`	The exact GPU model assigned to your notebook
`Total Memory (GB)`	Total VRAM available — a model must fit within this limit to run on this GPU
`Multiprocessors`	Number of streaming multiprocessors (SM) — each contains hundreds of CUDA cores

Step 5: Monitor GPU utilization in real-time

This step uses nvidia-smi with specific query flags to return a clean, structured summary of GPU utilization and memory usage. This is the same command used in production data centers to monitor GPU health during AI training runs.

```
!nvidia-smi --query-gpu=name,utilization.gpu,memory.used,memory.total --format=csv
```



Step 6: Create a tensor on the GPU

A tensor is the fundamental data structure used in AI — it is a multi-dimensional array of numbers, similar to a matrix. In this step, we create a large tensor directly on the GPU rather than on the CPU. This is equivalent to allocating data directly in GPU memory (VRAM), making it immediately available to the GPU's compute cores without any transfer delay.

```
device = "cuda"

x = torch.rand(10000, 10000, device=device)

print("Tensor Device:", x.device)
```

Expected output: The output will confirm that the tensor has been allocated on the GPU by displaying cuda:0 (or a similar CUDA device identifier) as its device. It will also print the tensor's dimensions as torch.Size([10000, 10000]), indicating a 10,000 by 10,000 matrix.

Re-run the nvidia-smi command from Step 5 now, you will see memory used has increased.

```
[8]
✓ 0s
    !nvidia-smi --query-gpu=name,utilization.gpu,memory.used,memory.total --format=csv

    name, utilization.gpu [%], memory.used [MiB], memory.total [MiB]
    Tesla T4, 0 %, 489 MiB, 15360 MiB
```

Step 7: Benchmark GPU matrix multiplication

Matrix multiplication is the core mathematical operation in neural network training and inference. Every time a model processes data through a layer, it is performing matrix multiplication. This step measures how long the GPU takes to multiply two large matrices together — giving you a direct sense of raw GPU compute speed.

torch.cuda.synchronize() is called before and after the operation. Note: GPU operations are asynchronous by default, meaning Python moves on before the GPU has finished. Synchronize forces Python to wait for the GPU to complete before reading the timer.

```
import time

device = "cuda"
A = torch.randn(5000, 5000, device=device)
B = torch.randn(5000, 5000, device=device)

torch.cuda.synchronize()
start = time.time()

C = torch.matmul(A, B)

torch.cuda.synchronize()
end = time.time()

print("Time:", round(end - start, 3), "seconds")
```

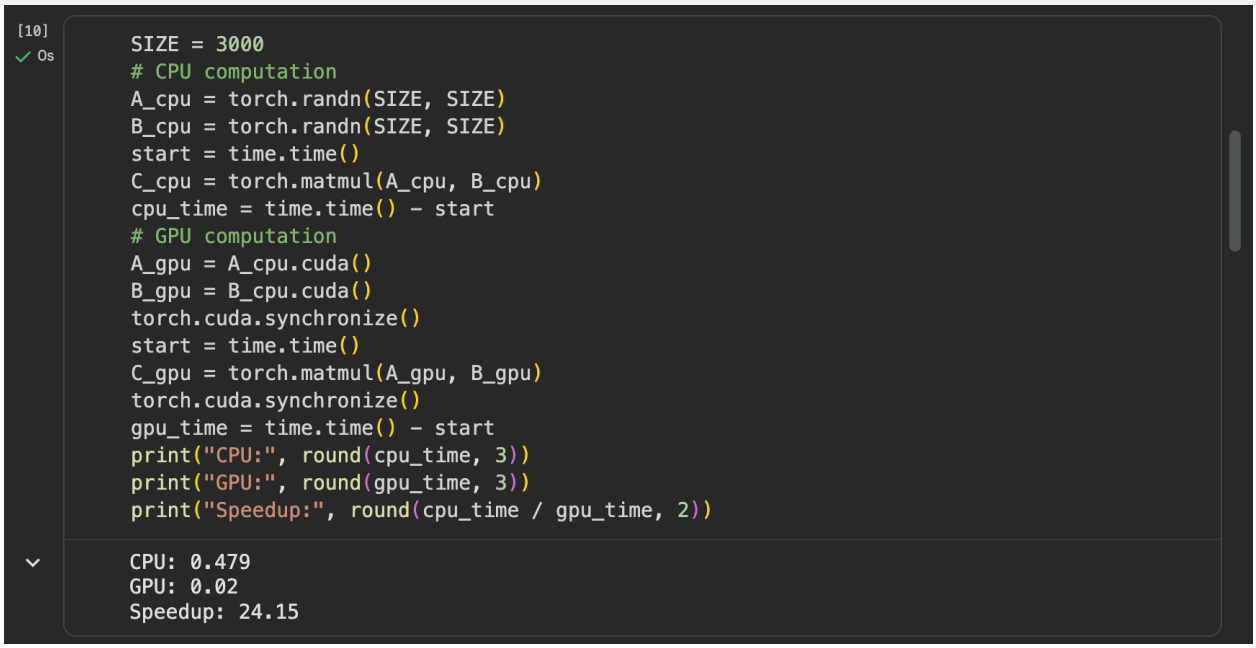
```
[9]
✓ 0s
    import time
    device = "cuda"
    A = torch.randn(5000, 5000, device=device)
    B = torch.randn(5000, 5000, device=device)
    torch.cuda.synchronize()
    start = time.time()
    C = torch.matmul(A, B)
    torch.cuda.synchronize()
    end = time.time()
    print("Time:", round(end - start, 3), "seconds")

    Time: 0.236 seconds
```

> **Note:** *The time will vary depending on the GPU model. An A100 will be faster than a T4. Note this number — you will compare it with CPU performance in the next step.*

Step 8: Compare CPU vs. GPU performance

You will run the same matrix multiplication on both the CPU and the GPU and directly compare the time taken. This demonstrates why GPUs are essential for AI, not because they are generally faster computers, but because they are specifically built to run many parallel mathematical operations simultaneously

```
SIZE = 3000

# CPU computation
A_cpu = torch.randn(SIZE, SIZE)
B_cpu = torch.randn(SIZE, SIZE)

start = time.time()
C_cpu = torch.matmul(A_cpu, B_cpu)
cpu_time = time.time() - start

# GPU computation
A_gpu = A_cpu.cuda()
B_gpu = B_cpu.cuda()

torch.cuda.synchronize()
start = time.time()
C_gpu = torch.matmul(A_gpu, B_gpu)
torch.cuda.synchronize()
gpu_time = time.time() - start

print("CPU:", round(cpu_time, 3))
print("GPU:", round(gpu_time, 3))
print("Speedup:", round(cpu_time / gpu_time, 2))
```



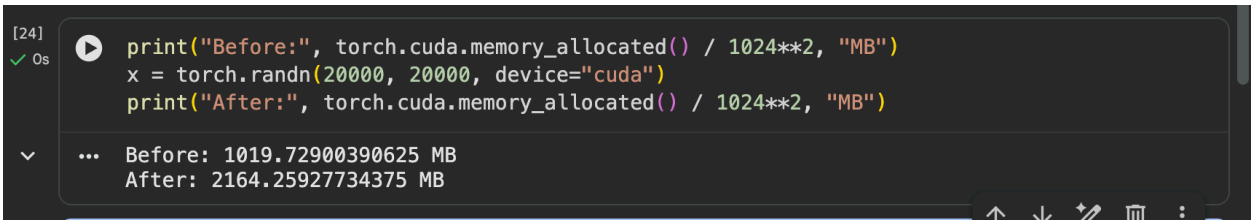
Expected Output:

Why is the GPU so much faster?

Step 9: Observe GPU memory allocation

In this step, you will check how GPU memory (VRAM) is consumed when you create tensors. Understanding memory allocation is critical in real AI work. if a model or dataset is too large to fit in VRAM, the training job will crash with an out-of-memory (OOM) error.

```
print("Before:", torch.cuda.memory_allocated() / 1024**2, "MB")

x = torch.randn(20000, 20000, device="cuda")

print("After:",  torch.cuda.memory_allocated() / 1024**2, "MB")
```



Expected Output: The before value shows the data that is still in memory from previous steps.

The increase in memory corresponds to a 20,000 × 20,000 matrix of 32-bit floats. Manually calculating the memory consumed= 20,000 × 20,000 × 4 bytes = 1,600,000,000 bytes ≈ 1,526 MB.

Step 10: Load a neural network model onto the GPU

In this step, you will build a small neural network that is a simplified version of a classifier (models for tasks like image classification). using PyTorch's nn.Sequential API and move it onto the GPU using .to(device). This is the standard pattern used in real AI workflows → define the model architecture, then send it to the GPU so all its computations happen there.

The model has three layers: a linear layer with 1,000 inputs and 512 outputs, a ReLU activation

```
import torch.nn as nn

device = "cuda"

model = nn.Sequential(
    nn.Linear(1000, 512),
    nn.ReLU(),
    nn.Linear(512, 10)
).to(device)

x = torch.randn(128, 1000, device=device)
out = model(x)
print(out.shape)
```

```
[25]
✓ 0s
    import torch.nn as nn
    device = "cuda"
    model = nn.Sequential(
    nn.Linear(1000, 512),
    nn.ReLU(),
    nn.Linear(512, 10)
    ).to(device)
    x = torch.randn(128, 1000, device=device)
    out = model(x)
    print(out.shape)

    torch.Size([128, 10])
```

Expected Output:

Output field	What it means
`128`	Batch size — the number of inputs processed simultaneously
`10`	Output classes — the model produces one score per class
`.to(device)`	Moves all model weights and biases into GPU VRAM

Step 11: Measure model inference time: CPU vs GPU

In this step, you will measure how long it takes to run inference (a forward pass) through the neural network on both CPU and GPU. This uses two different timing methods:

- time.perf_counter() for CPU timing — a high-resolution wall clock available in standard Python
- torch.cuda.Event for GPU timing — CUDA events are the correct way to time GPU operations because they are recorded on the GPU's own internal timeline, not the CPU clock

```
import torch

import torch.nn as nn
import time

# Model
model = nn.Sequential(
    nn.Linear(1000, 512),
    nn.ReLU(),
    nn.Linear(512, 1000)
)

x = torch.randn(128, 1000)

# CPU timing
start = time.perf_counter()
out_cpu = model(x)
cpu_time = (time.perf_counter() - start) * 1000

# GPU timing
device = torch.device("cuda")
model_gpu = model.to(device)
x_gpu = x.to(device)

torch.cuda.synchronize()
start_event = torch.cuda.Event(enable_timing=True)
end_event = torch.cuda.Event(enable_timing=True)

start_event.record()
out_gpu = model_gpu(x_gpu)
end_event.record()

torch.cuda.synchronize()
gpu_time = start_event.elapsed_time(end_event)

# Results
print(f"CPU time: {cpu_time:.3f} ms")
print(f"GPU time: {gpu_time:.3f} ms")
print(f"Speedup: {cpu_time / gpu_time:.1f}x")
print(f"Output shape: {out_gpu.shape}")
print(f"Memory used: {torch.cuda.memory_allocated()/1024**2:.1f} MB")
```

```
[26]
✓ 0s
import torch
import torch.nn as nn
import time
# Model
model = nn.Sequential(
nn.Linear(1000, 512),
nn.ReLU(),
nn.Linear(512, 1000)
)
x = torch.randn(128, 1000)
# CPU timing
start = time.perf_counter()
out_cpu = model(x)
cpu_time = (time.perf_counter() - start) * 1000
# GPU timing
device = torch.device("cuda")
model_gpu = model.to(device)
x_gpu = x.to(device)
torch.cuda.synchronize()
start_event = torch.cuda.Event(enable_timing=True)
end_event = torch.cuda.Event(enable_timing=True)
start_event.record()
out_gpu = model_gpu(x_gpu)
end_event.record()
torch.cuda.synchronize()
gpu_time = start_event.elapsed_time(end_event)
# Results
print(f"CPU time: {cpu_time:.3f} ms")
print(f"GPU time: {gpu_time:.3f} ms")
print(f"Speedup: {cpu_time / gpu_time:.1f}x")
print(f"Output shape: {out_gpu.shape}")
print(f"Memory used: {torch.cuda.memory_allocated()/1024**2:.1f} MB")

...  CPU time: 17.525 ms
     GPU time: 0.646 ms
     Speedup: 27.1x
     Output shape: torch.Size([128, 1000])
     Memory used: 407.8 MB
```

Step 12: Run a larger model

In Step 11 you measured inference time on a small model.

GPUs generally provide greater performance gains when there is more computation to perform. Modify the model and rerun your benchmark to investigate how the speedup changes.

Experiment with one or more of the following.

- Increase the size of the existing layers (e.g., more input, hidden, or output neurons)
- Increase the batch size (number of input samples processed at once)
- Increase the number of layers

Challenge: Can you achieve a speedup greater than **10×**? If so, describe which parameter changes had the greatest impact and explain why they improved GPU performance.